Query Performance Prediction Is Not Benefit Prediction: Observability for Selective Retrieval Interventions

Anonymous Author(s)

Abstract

Selective retrieval interventions, such as LLM query expansion (QE), cross-encoder reranking, and pseudo-relevance feedback (PRF), are often gated on a query performance predictor: intervene when retrieval looks like it is failing. We show why failure is the wrong target and measure by how much. Writing the decision as maximizing the expected gain of an intervention given the telemetry a system can observe, we decompose an oracle selector's gain exactly into gain on failing queries, the most a perfect failure detector can recover, and gain on healthy queries, which it never sees; with a per-query cost, a third term charges the cost spent on failures the intervention does not rescue. On TripClick and BEIR Natural Questions, across eight intervention, corpus, and retriever settings, this failure-only ceiling ranges from 8% to 57% of the oracle's gain, tracking how often an intervention rescues a failing query (1–46%) relative to how often it helps a healthy one. Detection itself is ordinary QPP (AUC 0.62–0.70, no better than NQC under BM25), and it beats predicting harm in seven of eight settings after Holm correction, though the gap largely vanishes on queries the intervention changes. Budget-matched routing confirms the limit: with pre-action telemetry, failure-, NQC-, and learned benefit-based routing all stay close to random routing and far below the oracle; benefit routing beats failure routing only for reranking on NQ. Telemetry observed after the intervention does better (on TripClick it cuts reranking's loss from −0.028 to −0.001 nDCG@10), but only once its cost is paid.

1 Introduction

Retrieval interventions help some queries and harm others. LLM query expansion (QE) can rewrite a query using the evidence it retrieves [13], yet it harms a measurable fraction of queries [1, 20]; cross-encoder reranking and pseudo-relevance feedback (PRF) are equally uneven. The natural response is to intervene selectively, and the natural gate is a query performance predictor (QPP): intervene when the current ranking looks like it is failing [2, 9]. That the gate's target differs from the quantity a policy needs is not new: selective-QE work predicts expansion benefit directly [7], and uplift modelling shows that targeting high-risk cases is not targeting the cases a treatment helps [3, 8]. What is missing is how much a failure gate gives up, why, and whether the telemetry a deployed system can observe, its *observability*, can do better.

Contributions. (C1) A decision framing of selective intervention under observable telemetry, with an exact decomposition of the *failure-only ceiling*, including a cost term (Section 3). **(C2)** Its measurement for LLM QE (BM25 and dense retrieval), reranking, and PRF on two corpora: the ceiling ranges from 8% to 57% of the oracle's gain and is explained by rescue and healthy-help rates. **(C3)** Paired, same-query AUCs of detection and action utility (benefit prediction) in all eight settings, Holm-corrected. **(C4)** Budget-matched routing that separates pre-action routing from post-action verification.

2 Related Work

QPP and selective QE. Pre- and post-retrieval QPP (Clarity [6], WIG [23], NQC [17]) predicts query difficulty. Cronen-Townsend et al. [7] decide whether to expand by comparing the clarity of expanded and unexpanded rankings; Amati et al. [2] gate expansion on difficulty predictors; Lv and Zhai [14] predict a per-query feedback weight; Collins-Thompson [5] trades expansion reward against risk. These works evaluate the gated system in aggregate; they neither bound what a failure gate can recover nor compare gates at matched budgets.

LLM QE and adaptive retrieval. Query2doc [19], MuGI [22], and CSQE [13] expand unconditionally; generative expansions often hurt strong retrievers [20] and fail on unfamiliar or ambiguous queries [1]. Kotte [10] gates prompt-only rewriting for dense retrievers (gate AUC 0.593, no reliable gain); Adaptive-RAG [9] routes on question complexity. Neither separates a gate's detection accuracy from its benefit prediction. In uplift terms [8], our setting is unusually clean: both outcomes are observed for every query offline, so the uplift label is exact up to relevance-label noise.

3 Framework

For a query q, let R_0 be the original ranking and R_a the ranking after intervention a, with metric M (nDCG@10). The intervention's effect is $\Delta_a(q) = M(R_a, q) - M(R_0, q)$ and its cost c_a ($\Delta_0 = c_0 = 0$). With full information the best decision is $a^*(q) = \arg\max_a\{\Delta_a(q) - \kappa c_a\}$, for a cost weight κ. A deployed controller sees only telemetry Z, so the best it can do is $\pi^*(Z) = \arg\max_a\{\mathbb{E}[\Delta_a \mid Z] - \kappa c_a\}$. What matters is therefore what Z reveals about Δ_a, not about $M(R_0)$. Two information sets differ in cost: *pre-action* telemetry Z_0 describes R_0 and is free; *post-action* telemetry Z_1 also describes R_a and exists only after c_a is paid. A failure gate replaces $\mathbb{E}[\Delta_a \mid Z_0]$ with $P(F \mid Z_0)$, where $F = \mathbf{1}[M(R_0, q) = 0]$.

The failure-only ceiling. Fix one intervention and write the cost per query as κ (nDCG@10 units). The oracle selector earns $G_{\text{orc}}(\kappa) = \mathbb{E}[(\Delta - \kappa)_+]$, and a *perfect* failure gate, which intervenes iff $F = 1$, earns $G_F(\kappa) = \mathbb{E}[(\Delta - \kappa)F]$. Because $M \geq 0$, a failing query cannot be harmed ($\Delta \geq 0$ when $F = 1$), and

$$G_{\text{orc}}(\kappa) - G_F(\kappa) = \underbrace{\mathbb{E}[(\Delta - \kappa)_+(1 - F)]}_{\text{gain on healthy queries}} + \underbrace{\mathbb{E}[(\kappa - \Delta)_+ F]}_{\text{cost on unrescued failures}} .$$

At $\kappa = 0$ the second term vanishes and the gate's share of the oracle is $\mathbb{E}[\Delta_+ F]/\mathbb{E}[\Delta_+]$. The identity is elementary; its use is that both terms are measurable offline for any intervention and bound what any failure detector, however accurate, can deliver.

4 Setup

Data. TripClick [15]: 1.52M health documents, the TAIL bucket's validation and test splits (1,175 queries each), with click-derived relevance labels. BEIR NQ [11, 18]: 2.68M passages, human labels; 800 validation and a disjoint 2,052-query test set.

Interventions. *LLM QE*: BM25 over SQLite FTS5 (Porter stemming, title weight tuned on validation); a deterministic rule picks one of four expansion operators or none (1.5%/1.9% of queries under BM25, 10.6%/5.8% dense), and `qwen2.5:3b` (4-bit, greedy, ≤ 2 calls) writes a corpus-grounded expansion. On TripClick and under dense retrieval (`bge-small-en-v1.5` [21]) the expanded ranking's top candidates are reranked by a MiniLM cross-encoder [16]; the original is not, so those labels include the reranker's effect. TripClick's deployed QE fuses original and expanded rankings (RRF); NQ's uses the expanded ranking. *Reranking*: the cross-encoder scores the BM25 top 100; its top 50 lead, the BM25 order follows. *PRF*: RM3-style [12] weights from the top d documents select t terms whose BM25 ranking is interpolated with the original (weight λ); (d, t, λ) is tuned on validation over 108 settings ($(20, 10, 0.9)$ TripClick, $(20, 30, 0.6)$ NQ). Reranking and PRF are applied to every BM25 query.

Labels and models. *Failure*: $F = 1$ (45% of TripClick and 49% of NQ test queries under BM25). *Harmed*: $\Delta < 0$, ties not harmed. Detection is a logistic regression on 8 pre-action features (score shape, term coverage, rare-term IDF, entity overlap, coherence, perturbation stability; 6 under dense). Action utility is fit on Δ from post-action features (for LLM QE: changes in margin, coverage, coherence, rewrite drift, reranker agreement, overlap; linear under BM25, a logistic harm model under dense; for reranking and PRF: the 8 pre-action features plus overlap, top-1 change, rank-biased overlap, and the new ranking's score shape). For pre-action routing, a *benefit* model regresses Δ on the 8 pre-action features alone. Every model and threshold is fit on validation and scored once on test; intervals are paired bootstraps over queries (10^4 resamples).

Evidence ledger. No result here is independent confirmation: every NQ query and the TripClick TAIL test set have been inspected. *Fixed before a test run*: the NQ BM25 and dense runs, and the reranking and PRF runs, each under a written scope. *Pre-committed analyses of already-inspected traces*: the AUCs and policies (Sections 6 and 7), the reranker-matched relabelling, and all of Section 5, budget routing, and Holm correction, which recombine frozen traces without new runs. *Exploratory*: TripClick test was used during development (an early verifier was calibrated and two apparatus defects were found on it), and the changed-query columns of Table 2 were added after seeing the main results. Pre-commitments and scripts are in the anonymized repository: https://anonymous.4open.science/r/agentic-query-expansion-8939/.

5 The Failure-Only Ceiling

A perfect failure detector would capture between 8% and 57% of the oracle's gain (Table 1), even though, by the identity, it never makes a query worse. The share follows how often an intervention rescues a failing query relative to how often it helps a healthy one. Reranking rescues often (17% and 46% of failing queries, against 24% and 55% of healthy ones helped), so half its headroom lies in failing queries. LLM QE rescues 6–15% of failing queries and helps 15–25% of healthy ones (a third to two-fifths). PRF on TripClick almost never rescues (0.8%, against 12%), so nearly all its headroom is in healthy queries. The rest of the oracle's gain sits in healthy queries that the same intervention also harms often (19–56%), which is why separating them needs a benefit prediction rather than a failure one.

Table 1: Failure-only ceiling (test). Rescue: $P(\Delta > 0 \mid F)$. Healthy helped / harmed: $P(\Delta \gtrless 0 \mid \neg F)$. Share: perfect failure gate's gain over the oracle's at $\kappa = 0$, 95% CI.

Setting	$P(F)$	Rescue	Healthy help / harm		Share
LLM QE, TripClick	.45	.07	.15	.19	42% [32, 52]
LLM QE, NQ	.49	.07	.19	.33	33% [26, 40]
LLM QE, TripClick, dense	.59	.06	.25	.41	34% [25, 43]
LLM QE, NQ, dense	.26	.15	.19	.29	33% [27, 40]
Reranking, TripClick	.45	.17	.24	.56	52% [44, 60]
Reranking, NQ	.49	.46	.55	.19	57% [54, 60]
PRF, TripClick	.45	.01	.12	.20	8% [1, 17]
PRF, NQ	.49	.05	.20	.22	32% [25, 39]

Costs widen the gap. At $\kappa = 0.02$ nDCG@10 per intervention, the second term (cost on unrescued failures) reaches 0.004–0.011, and the perfect failure gate's net gain turns negative for PRF on both corpora and for dense TripClick QE (-0.008, -0.001, -0.002), while the oracle keeps 0.011, 0.025, and 0.027. A real detector falls below these ceilings.

6 Detection vs. Action Utility

Detection is ordinary QPP. Detection AUC is 0.62–0.70. NQC over the top-100 BM25 scores [17] matches our detector on TripClick (0.647 vs. 0.649) and beats it on NQ (0.721 vs. 0.680); WIG [23] is weaker. The relevant-document count alone predicts failure at only 0.570 and 0.521, and the detector stays significant within count strata.

Detection beats harm prediction, structurally. Detection beats harm AUC in seven of eight settings after Holm correction (Table 2), and the gaps persist when harm requires $\Delta < -0.05$ or -0.10 in six of eight. They are partly structural: failing queries cannot be harmed and are mostly left unchanged, so the harm label is itself largely a failure indicator. Restricted to queries the intervention changed, detection drops to 0.47–0.58 and every BM25 gap vanishes; only dense NQ keeps one. Dropping failing queries raises TripClick reranking's harm AUC to 0.659, level with detection. Predicting *help* is easier where the intervention helps often (NQ reranking: 0.679, tied with detection). Two controls hold: a harm model on pre- plus post-action features still trails detection (TripClick BM25 +0.040 [+0.003, +0.078], dense NQ +0.070), and relabelling TripClick QE against the reranked original leaves a gap of +0.059 [+0.012, +0.107] (the dense runs stored no reranked original). The action-utility score does rank queries by gain for LLM QE and reranking (best-linear-predictor slopes [4] +0.42 to +1.21, CIs exclude zero) but not for PRF or dense NQ.

7 Routing at Matched Budgets

Pre-action routing. At matched budgets, every scorer that decides before intervening stays close to random routing and far below the oracle (Table 3, Figure 1). Failure routing beats random by at most 3.2×10^{-3} and falls below it for PRF on NQ; NQC is no better than the learned detector (no difference survives Holm). A pre-action benefit model beats failure routing only for reranking on NQ (+0.009 [+0.003, +0.015]). This is the framework's prediction:

Table 2: Detection vs. action-utility AUC on the same test queries (LLM QE: acted-upon queries; reranking, PRF: all). Harm, Helped: action-utility AUC for harmed-vs.-rest and helped-vs.-rest. Diff.: paired bootstrap of detection minus harm AUC (95% CI); † not significant after Holm correction over the eight. Last column: detection minus direction (helped vs. harmed) on queries the intervention changed ($\Delta \neq 0$; exploratory).

Intervention	Corpus	Retriever	n	Detection	Harm	Diff.	Helped	Diff., $\Delta \neq 0$
LLM QE	TripClick	BM25	1157	0.643	0.552	+0.091 [+0.045, +0.136]	0.540	−0.044 [−0.127, +0.039]
LLM QE	NQ	BM25	2013	0.675	0.556	+0.119 [+0.076, +0.162]	0.599	−0.041 [−0.122, +0.036]
LLM QE	TripClick	dense	1050	0.621	0.599	+0.022 [−0.028, +0.072]†	0.464	+0.043 [−0.063, +0.145]
LLM QE	NQ	dense	1932	0.695	0.611	+0.084 [+0.045, +0.123]	0.406	+0.122 [+0.047, +0.197]
Reranking	TripClick	BM25	1175	0.649	0.575	+0.075 [+0.029, +0.119]	0.513	+0.012 [−0.066, +0.089]
Reranking	NQ	BM25	2052	0.680	0.514	+0.166 [+0.120, +0.213]	0.679	−0.025 [−0.079, +0.027]
PRF	TripClick	BM25	1175	0.649	0.572	+0.077 [+0.014, +0.140]	0.393	+0.001 [−0.279, +0.433]
PRF	NQ	BM25	2052	0.680	0.501	+0.179 [+0.131, +0.227]	0.553	−0.045 [−0.140, +0.048]

Table 3: Pre-action routing: nDCG@10 gain over the original ($\times 10^3$), averaged over budgets of 10, 25, 50, 75, and 100% of queries intervened on. Random: expected gain. Last column: benefit minus failure routing, 95% CI; * significant after Holm over the six settings.

Setting	Fail.	NQC	Benefit	Rand.	Oracle	Ben.−Fail.
LLM QE, TripClick	1.7	−0.5	0.6	0.3	19.7	−1.2 [−3.6, +1.4]
LLM QE, NQ	−8.1	−9.1	−7.5	−11.3	28.4	+0.6 [−2.0, +2.9]
Rerank, TripClick	−13.1	−15.4	−11.3	−14.4	40.5	+1.8 [−3.7, +6.3]
Rerank, NQ	118.7	114.3	127.5	117.0	199.5	+8.8 [+3.3, +15.0]*
PRF, TripClick	−0.4	−0.4	−0.3	−0.6	9.6	+0.1 [−1.0, +1.0]
PRF, NQ	4.5	3.8	4.9	5.0	24.0	+0.4 [−1.7, +2.4]

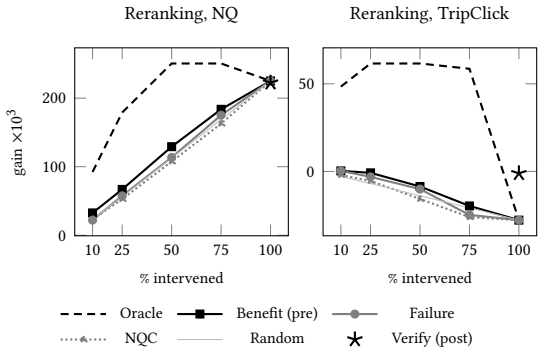


Figure 1: nDCG@10 gain over BM25 when the top-k queries ranked by each pre-action scorer are reranked. The star is post-action verification, which reranks every query and keeps those with positive predicted gain.

Z_0 describes R_0, and R_0 reveals little about Δ_a beyond whether $M(R_0) = 0$. For LLM QE, the deployed failure-selective policy (median threshold, 49–51% routed) halves generator calls; on NQ it recovers most of unconditional QE's loss (+0.015 [+0.008, +0.022] over Static QE, residual −0.007 against BM25), and on TripClick no QE policy, including a closed-loop agent, differs from BM25 (MDEs 0.007–0.011).

Post-action verification. With Z_1, a controller can compute the intervention and keep it only if the predicted gain is positive; its compute is 100%. For reranking on TripClick, where reranking every query loses −0.028 [−0.042, −0.013], verification keeps 15% and loses only −0.001 [−0.005, +0.003], while the failure gate still loses −0.010. On NQ it keeps 94% of reranked results (+0.223 vs. +0.225) and 67% of PRF results (+0.0085 vs. +0.0097). Post-action telemetry carries information about Δ_a that pre-action telemetry lacks, at the price of paying for every intervention. Reranking numbers use the current reranker input; the stored TripClick arm used for the relabelling predates a fix to that text (0.285 vs. 0.287).

8 Discussion and Limitations

Failure prediction is the most accurate signal a controller has, and it targets the wrong quantity: the gain a failure gate can reach is capped by how often an intervention rescues failing queries, and that cap differs nearly sevenfold across interventions. The difficulty is not counterfactual missingness, since both outcomes are observed offline, nor the generator alone, since reranking and PRF show the same gaps; a likelier common factor is that pre-action telemetry describes the original ranking, while Δ_a depends on what the intervention retrieves.

Limitations. One small (3B) generator; reranking and PRF only under BM25, with one cross-encoder and one PRF family. Two corpora; under dense retrieval, retriever strength and domain are confounded, and the dense QE labels include the reranker. TripClick's labels are click-derived and sparse. The cost weight κ is illustrative, not measured. The TripClick QE benefit model is fit to the unfused validation label, since no fused validation ranking exists. All analyses reuse inspected test sets; every pre-committed quantity is reported in full in the repository.

9 Conclusion

Detecting that retrieval has failed is ordinary QPP, and it bounds what a selective controller can gain: a perfect failure detector would capture 8–57% of an oracle's gain across LLM QE, reranking, and PRF, depending on how often each intervention rescues a failing query. Pre-action telemetry, learned or not, routes little better than chance at matched budgets; post-action telemetry separates helped from harmed queries better, but only after the intervention is paid for. Selective-intervention work should report the failure-only ceiling and budget-matched routing alongside any detector's AUC.

Ethical Considerations

All data used (TripClick, BEIR Natural Questions) are existing research benchmarks, used under their licenses; TripClick is used for

non-commercial research, and no model or per-query output fit on it is released. TripClick's click logs are aggregate, de-identified interactions, and we make no attempt to re-identify searchers. No human-subjects data was collected. The generator is a small, locally hosted model used only to rewrite queries within a bounded action space. Our central finding cautions against deploying generative interventions on the strength of a failure detector alone.

References

[1] Kenya Abe, Kunihiro Takeoka, Makoto P. Kato, and Masafumi Oyamada. 2025. LLM-based Query Expansion Fails for Unfamiliar and Ambiguous Queries. *arXiv preprint arXiv:2505.12694* (2025). https://arxiv.org/abs/2505.12694

[2] Giambattista Amati, Claudio Carpineto, and Giovanni Romano. 2004. Query Difficulty, Robustness, and Selective Application of Query Expansion. In *Advances in Information Retrieval: 26th European Conference on IR Research (ECIR) (LNCS, Vol. 2997)*. 127–137. doi:10.1007/978-3-540-24752-4_10

[3] Eva Ascarza. 2018. Retention Futility: Targeting High-Risk Customers Might Be Ineffective. *Journal of Marketing Research* 55, 1 (2018), 80–98. doi:10.1509/jmr.16.0163

[4] Victor Chernozhukov, Mert Demirer, Esther Duflo, and Iván Fernández-Val. 2018. Generic Machine Learning Inference on Heterogeneous Treatment Effects in Randomized Experiments, with an Application to Immunization in India. *arXiv preprint arXiv:1712.04802* (2018). https://arxiv.org/abs/1712.04802

[5] Kevyn Collins-Thompson. 2009. Reducing the Risk of Query Expansion via Robust Constrained Optimization. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 837–846. doi:10.1145/1645953.1646059

[6] Steve Cronen-Townsend, Yun Zhou, and W. Bruce Croft. 2002. Predicting Query Performance. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 299–306. doi:10.1145/564376.564429

[7] Steve Cronen-Townsend, Yun Zhou, and W. Bruce Croft. 2004. A Framework for Selective Query Expansion. In *Proceedings of the Thirteenth ACM International Conference on Information and Knowledge Management*. 236–237. doi:10.1145/1031171.1031220

[8] Carlos Fernández-Loría and Foster Provost. 2022. Causal Classification: Treatment Effect Estimation vs. Outcome Prediction. *Journal of Machine Learning Research* 23, 59 (2022), 1–35.

[9] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*. https://aclanthology.org/2024.naacl-long.389/

[10] Varun Kotte. 2026. Not All Queries Need Rewriting: When Prompt-Only LLM Refinement Helps and Hurts Dense Retrieval. *arXiv preprint arXiv:2603.13301* (2026). https://arxiv.org/abs/2603.13301

[11] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, et al. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. doi:10.1162/tacl_a_00276

[12] Victor Lavrenko and W. Bruce Croft. 2001. Relevance-Based Language Models. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 120–127. doi:10.1145/383952.383972

[13] Yibin Lei, Yu Cao, Tianyi Zhou, Tao Shen, and Andrew Yates. 2024. Corpus-Steered Query Expansion with Large Language Models. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*. 393–401.

[14] Yuanhua Lv and ChengXiang Zhai. 2009. Adaptive Relevance Feedback in Information Retrieval. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 255–264. doi:10.1145/1645953.1645988

[15] Navid Rekabsaz, Oleg Lesota, Markus Schedl, Jon Brassey, and Carsten Eickhoff. 2021. TripClick: The Log Files of a Large Health Web Search Engine. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2507–2513. doi:10.1145/3404835.3463242

[16] Sentence Transformers. 2026. cross-encoder/ms-marco-MiniLM-L6-v2. https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2

[17] Anna Shtok, Oren Kurland, David Carmel, Fiana Raiber, and Gad Markovits. 2012. Predicting Query Performance by Query-Drift Estimation. *ACM Transactions on Information Systems* 30, 2 (2012), 11:1–11:35. doi:10.1145/2180868.2180873

[18] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. https://arxiv.org/abs/2104.08663

[19] Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query Expansion with Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 9414–9423. doi:10.18653/v1/2023.emnlp-main.585

[20] Orion Weller, Kyle Lo, David Wadden, Dawn Lawrie, Benjamin Van Durme, Arman Cohan, and Luca Soldaini. 2024. When do Generative Query and Document Expansions Fail? A Comprehensive Study Across Methods, Retrievers, and Datasets. In *Findings of the Association for Computational Linguistics: EACL 2024*. 1987–2003.

[21] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2023. C-Pack: Packed Resources For General Chinese Embeddings. *arXiv preprint arXiv:2309.07597* (2023). https://arxiv.org/abs/2309.07597

[22] Le Zhang, Yihong Wu, Qian Yang, and Jian-Yun Nie. 2024. Exploring the Best Practices of Query Expansion with Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 1872–1883. doi:10.18653/v1/2024.findings-emnlp.103

[23] Yun Zhou and W. Bruce Croft. 2007. Query Performance Prediction in Web Search Environments. In *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 543–550. doi:10.1145/1277741.1277835